

SE333 Final Project — Reflection Report

Introduction

This project applied a coverage-first, iterative workflow to improve automated test coverage for the Spring Petclinic application. The primary objective was to identify low-coverage classes, write targeted unit tests, and verify measurable improvements at each iteration using JaCoCo and a custom MCP coverage heatmap tool. The project began with most controller and system classes at 0% line coverage. Through multiple iterations across feature branches, coverage increased to between 90–100% across nearly all concrete classes.

Methodology

The workflow followed these steps during each iteration:

1. Run `mvn clean test jacoco:report` to generate a fresh JaCoCo XML report.
2. Invoke the `coverage_heatmap` MCP tool to rank classes by line coverage (lowest first).
3. Select the lowest-covered concrete class.
4. Write focused unit tests using JUnit 5 and Mockito (no Spring context required).
5. Apply Spring Java Format (`mvn spring-javaformat:apply`) before re-running tests.
6. Re-measure coverage and commit improvements to a dedicated feature branch.
7. Push the branch and open a pull request on GitHub.

All tests were written as pure Mockito-based unit tests using `MockitoAnnotations.openMocks(this)`. This avoided brittle Spring context startup while still exercising controller, formatter, and configuration branch paths.

Results and Discussion

Starting coverage (pre-project):

- PetController, VisitController, VetController, and most system/configuration classes: 0%

Final coverage achieved:

- OwnerController: 96.23%
- PetController: 90.91%
- VisitController: 90.91%
- PetTypeFormatter: 90.91%
- VetController: 100%
- WelcomeController: 100%
- WebConfiguration: 100%
- CacheConfiguration: 100%
- CrashController: 100%
- PetClinicRuntimeHints: 100%

- Vet, Specialty, Vets (domain models): 100%
- PetClinicApplication: 66.67%

Repository interfaces (OwnerRepository, VetRepository, PetTypeRepository) remain at 0% by design because they contain no concrete method bodies for JaCoCo to instrument in unit-level reports.

The largest coverage gains came from controller tests, particularly **OwnerController** and **PetController**, which required careful fixture setup to test both the *new entity* and *persisted entity* code paths. Several system and configuration classes required only 1–3 targeted tests to reach 100% coverage.

AI-Assisted Development Insights

AI (GitHub Copilot via the MCP workflow) was most useful for rapid test scaffolding and boilerplate generation, expanding branch coverage by suggesting additional edge-case paths, and sequencing build commands and diagnosing format validation failures

Manual judgment was still required for determining which test assertions provided meaningful quality beyond simple coverage, understanding entity lifecycle differences between new vs. persisted objects in Spring Data and deciding when a practical coverage ceiling had been reached (for example, the **PetClinicApplication** main method).

The heatmap + AI loop proved effective because every iteration produced objective numeric feedback. This made it easy to identify the next class to target and quickly determine whether a new test added real value.

Challenges and Debugging

Several technical challenges emerged during development including: **Spring Java Format enforcement** — The build failed repeatedly until `mvn spring-javaformat:apply` was added as a required step before each `mvn test` execution. **Owner/Pet/Visit fixture setup** — Pet ID and ownership semantics in Spring Petclinic differ between newly created and already persisted entities, requiring careful test fixture design. **MockedStatic for PetClinicApplication** — Using `MockedStatic<SpringApplication>` to cover the application entry point required Mockito 5 and careful static mock scoping. **Branch-level vs. line-level gaps** — Some classes showed high line coverage but lower branch coverage due to null-check paths that were difficult to trigger in isolation.

Recommendations for Future Work

Future improvements could include:

1. Adding mutation testing (PIT) to verify that tests detect real defects rather than simply executing lines of code.
2. Introducing CI coverage gates (for example, requiring a minimum of 80% line and branch coverage).
3. Expanding integration tests for repository and data-layer behavior where meaningful.
4. Using parameterized tests to reduce duplication across similar test paths.